

2026

Vendor Machine Marketing Analysis



Tong Pak Yuen Isaac NG Ching Hei Fu Lok Tim

IT114116

2026/6/29

Table of Content

Introduction.....	2
Data	3
Setting and Utilising GitHub	4
Roles	5
Product Backlog items	5
Sprint Backlog items.....	6
Code solution and Explanation	7
Evaluation	29
Conclusion	33

Introduction

Vending machines combine the concept of online business management with offline convenience services. By adopting a compact, self-service operating model, they significantly reduce labor costs and make affordable, high-quality products readily accessible to consumers. This approach has emerged as another mainstream retail format in recent years. For vending machine operators, key decisions revolve around product replenishment frequency, category selection, supply quantities, and optimal machine placement locations. Scientific business data analysis plays a crucial role here: it helps operators gain deeper insight into user needs, accurately predict product demand, deliver more precise and attentive services, and guide strategic business direction—making it an essential driver for the ongoing development and success of the vending machine retail model.

Data

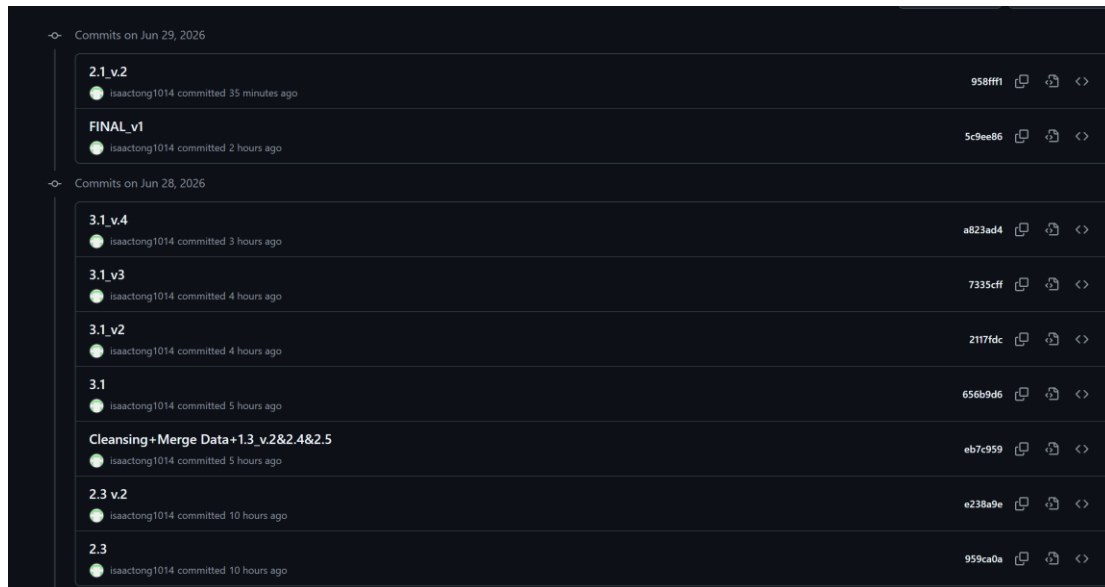
The only dataset is from the Vendor. The dataset includes transaction records and product information. The following is the metadata table.

Field (Chinese Field Name)	Type	Description	range	Sample
Order ID (订单号)	String	Unique identification of each transaction	Unique string	
Machine ID (设备 ID)	String	The vendor machine that makes the order	Unique string	
Payable Amount (应付金额)	Double	Total amount of the order that has to be paid	--	4.5
Actual Paid (实际金额)	Double	Actual paid amount	--	4.5
Product (商品)	String	The purchased product	--	營養快線
Location (地点)	String	The Vendor Machine Location	A to E	A
Payment Time(支付时间)	Time Stamp	The DateTime that the payment was made.	In Year 2017	1/1/2017 0:53
Transaction Status (状态)	String	The order status after payment.the term “已出貨未退貨”means the order was done and no refund required.	--	已出貨未退貨
Withdraw (提现)	String	The indicator that the vendor already received the payment. The term “已提现” means the payment already withdrawn by the vendor.	--	已提现
Category (大类)	String	The categorized label of the product	饮料, 非饮料	非饮料
Sub-Category (二级类)	String	The sub-categorized label of the product	--	饼干糕点

Setting and Utilising GitHub

Provide the link and screenshots of your GitHub project, and also some screen captures, including tree views and document changes.

<https://github.com/isaactong1014/ITP4475-group-project>



```
1  # @ -9,8 +9,8 @@ require(ggplot2)
2
3  require(mclust)
4
5  require(rpart)
6
7  require(rpart.plot)
8
9  + require(foreign)
10
11 + require(e1071)
12
13 ---
14
15
16 data input
17
18 @@ -63,8 +65,9 @@ combine
19
20
21 Mission 1.1 According to the merged dataset, suppose each location only have one vendor machine. Extract the Sales record from the merged dataset and store the data separately based on the Vendor machine location
22 "A", "B", "C", "D", "E".
23
24 '''(r)
25
26 sales_by_location <- sqldf("select * from combine
27
28 -         order by 地点")
29
30 +         order by 地点")
31
32 sales_by_location
33
34 +
35
36 ---
37
38
39 Mission 1.2 Calculate each vending machine's sales revenue and number of orders for April 2017, and also compute the overall total sales revenue and total number of orders across all vending machines. Display the
40 results in a table within the report.
41
42
43 @@ -125,23 +128,23 @@ loc_E
44
45
46 ggplot(loc_A, aes(x = Month, y = Diff)) +
47
48   geom_bar(stat = "identity",
49
50           show.legend = FALSE) +
51
52 -   labs(title = "Growth Rate of Vendor Machine A", y = "Monthly Revenue", x = "Month")
53
54 +   labs(title = "Growth Rate of Vendor Machine A", y = "Growth Rate", x = "Month")
55
56 ggplot(loc_B, aes(x = Month, y = Diff)) +
57
58   geom_bar(stat = "identity",
```

Roles

Name	Role	Responsibilities
Tong Pak Yuen Isaac	Leader	Coding
NG Ching Hei	Cooperator	Report writing
Fu Lok Tim	Cooperator	Report writing

Product Backlog items

Sprint	Task	Category	priority	status
1	Uploading data to R	DATA	LOW	Under Review
1	Data Cleansing	DATA	HIGH	Under Review
2	Relationship between price and bedroom numbes	Modelling	Middle	Comitted

Sprint Backlog items

Role	Tasks	Owner	Status	Estimated effort	Day 1	Day 2	Day 3	Day 4	Day 5	Day 6	Day 7
Leader	R coding	Tong Pak Yuen Isaac	Completed	12	2	3	4	2	1	0	0
Cooperator	Report writing (evaluation & conclusion)	NG Ching Hei	Completed	12	0	0	0	4	3	3	2
Cooperator	Report writing (code explanation)	Fu Lok Tim	Completed	12	0	0	4	4	3	1	0

Day	Detail	Owner
1	Study the data Data input Cleansing	Tong Pak Yuen Isaac
2	Merge data Mission 1.1 - 1.3	Tong Pak Yuen Isaac
3	Mission 2.1 - 2.3	Tong Pak Yuen Isaac
	Explanation Data input, Cleansing, Merge data, 1.1 - 1.3	Fu Lok Tim
4	Mission 2.4 - 2.5	Tong Pak Yuen Isaac
	Explanation 2.1 - 2.4	Fu Lok Tim
	Evaluation Q1 - 2	NG Ching Hei
5	Mission 3.1 - 3.2	Tong Pak Yuen Isaac
	Explanation 2.5 - 3.2	Fu Lok Tim
	Evaluation Q3	NG Ching Hei
6	Finalized	Fu Lok Tim
	Evaluation Q4	NG Ching Hei
7	Conclusion	NG Ching Hei

Code solution and Explanation

Read the dataset and **combine** the two sheets into a single dataframe.

Code: <pre>sale_data <- read.xlsx("Sale_Data.xlsx","Sales_Record") product_information <- read.xlsx("Sale_Data.xlsx","Product_Information") names(product_information)[names(product_information) == '商品'] <- '商品 1' combine <- sqldf("select * from sale_data inner join product_information on (sale_data.商品 = product_information.商品 1)")</pre>	Key steps: 1. Load the ("Sales_Record") sheet from the Excel file into sale_data dataframe 2. Load the ("Product_Information") sheet from the Excel file into product_information dataframe 3. Rename product column in ("product_information") to '商品 1' to avoid conflict 4. Merge both sheets using INNER JOIN on product name
--	--

Explanation...

The “Sales_Record” sheet contains 70,680 transaction records with details like order ID, product, amount, location, and payment time. The “Product_Information” sheet provides product categories and sub-categories. An INNER JOIN merges both datasets on the product name, creating a combined dataframe with all transaction details plus product categorization. This merged dataset serves as the foundation for all subsequent analysis tasks.

Convert the payment datetime to standard datetime format.

Code: Sys.setenv(TZ = "UTC") sale_data\$支付时间 <- as.POSIXct(sale_data\$支付时间, format = "%Y-%m-%d %H:%M:%S", tz = "UTC")	Key steps: 1. Set system timezone to UTC for consistencySpecify structure. 2. Convert payment time column from character to POSIXct datetime format. 3. Specify the input format to match the data structure
---	---

Explanation...

The payment time is originally stored as character strings in "YYYY-MM-DD HH:MM:SS" format. Converting to POSIXct datetime format enables proper time-based operations such as filtering by month, extracting day/month for aggregations, calculating time differences, and chronological sorting. The UTC time zone ensures consistency across all time-based calculations regardless of system locale.

Convert all payment-related columns to calculable values

Code: <pre>checking_price <- sqldf("select sale_data.订单号, sale_data.商品, sale_data.应付金额, sale_data.实际金额 from sale_data where 应付金额 != 实 际金额")</pre>	Key steps: 1. Identify any records where payable amount differs from actual paid amount 2. Confirm both amount columns contain numeric data types 3. Verify data integrity before performing financial calculations
--	---

Explanation...

The payable amount and actual paid amount are already stored as numeric/double values in the dataset, making them directly usable for mathematical operations like summing, averaging, and aggregation. The verification query checks for any discrepancies between these two columns to identify potential data quality issues. In a clean dataset, these values should match, confirming accurate financial records for revenue analysis.

Prove that all order ID are unique and there are 70680 unique salse records

Code: <pre>count <- nrow(sale_data) paste("There are", count, "sales records.") checking_record <- sqldf("select DISTINCT sale_data.订单号 from sale_data")</pre>	Key steps: 1. Count total number of rows in sale_data 2. Display the total count 3. Query distinct order IDs to verify uniqueness 4. Compare both counts to confirm no duplicate order IDs exist
---	---

Explanation...

Data integrity is verified by confirming that the total number of records “70680” matches the count of distinct order IDs. This ensures that each transaction is uniquely identified with no duplicate records, which is essential for accurate sales volume and revenue calculations in subsequent analysis.

Mission 1.1

Code:

sales_by_location <- sqldf("select * from combine
order by 地点")

sales_by_location

Partial / Complete Result Screenshot:

1-10 of 70,680 rows | 1-10 of 12 columns

订单号	设备ID	应付...	实际...	商品	支付时间	地点	状态	提现	商品1
DD201708167493663555814061164	E43A6E078A04172	3.0	3.0	40g双汇玉米热狗肠	2017-01-01 09:33:00	A	已出货未...	已...	40g双汇玉米热狗肠
DD201708167493663386737478695	E43A6E078A04172	5.8	5.8	100g卫龙点心面黑椒牛排味	2017-01-02 03:28:59	A	已出货未...	已...	100g卫龙点心面黑椒牛排味
DD201708167493663503095993712	E43A6E078A04172	0.8	0.8	咪咪虾条马来西亚风味	2017-01-02 03:33:00	A	已出货未...	已...	咪咪虾条马来西亚风味
DD2017081110292488D71D62358F9	E43A6E078A04172	3.0	3.0	250ml维他柠檬茶	2017-01-02 03:54:59	A	已出货未...	已...	250ml维他柠檬茶
DD201708167493507701935975302	E43A6E078A04172	3.5	3.5	东鹏特饮	2017-01-02 04:18:00	A	已出货未...	已...	东鹏特饮
DD2017081018210213E80D235B1C8	E43A6E078A04172	3.0	3.0	250ml维他柠檬茶	2017-01-02 04:25:00	A	已出货未...	已...	250ml维他柠檬茶
DD2017081018210213E80D235B1C8	E43A6E078A04172	3.0	3.0	卫龙大面筋	2017-01-02 04:25:00	A	已出货未...	已...	卫龙大面筋
DD2017081110281595114AF30BC0D	E43A6E078A04172	3.5	3.5	东鹏特饮	2017-01-02 16:50:00	A	已出货未...	已...	东鹏特饮
DD201708167493662937167036721	E43A6E078A04172	1.5	1.5	卫龙亲嘴烧红烧牛肉味	2017-01-02 20:01:59	A	已出货未...	已...	卫龙亲嘴烧红烧牛肉味
DD2017052614372337D05D9082898	E43A6E078A04172	4.5	4.5	阿萨姆奶茶	2017-01-02 21:34:00	A	已出货未...	已...	阿萨姆奶茶

1-10 of 70,680 rows | 1-10 of 12 columns

1-10 of 70,680 rows | 3-12 of 12 columns

应付金额	实际金额	商品	支付时间	地点	状态	提现	商品1	大类	二级类
3.0	3.0	40g双汇玉米热狗肠	2017-01-01 09:33:00	A	已出货未退款	已提现	40g双汇玉米热狗肠	非饮料	肉干/豆制品/蛋
5.8	5.8	100g卫龙点心面黑椒牛排味	2017-01-02 03:28:59	A	已出货未退款	已提现	100g卫龙点心面黑椒牛排味	非饮料	肉干/豆制品/蛋
0.8	0.8	咪咪虾条马来西亚风味	2017-01-02 03:33:00	A	已出货未退款	已提现	咪咪虾条马来西亚风味	非饮料	膨化食品
3.0	3.0	250ml维他柠檬茶	2017-01-02 03:54:59	A	已出货未退款	已提现	250ml维他柠檬茶	饮料	茶饮料
3.5	3.5	东鹏特饮	2017-01-02 04:18:00	A	已出货未退款	已提现	东鹏特饮	饮料	功能饮料
3.0	3.0	250ml维他柠檬茶	2017-01-02 04:25:00	A	已出货未退款	已提现	250ml维他柠檬茶	饮料	茶饮料
3.0	3.0	卫龙大面筋	2017-01-02 04:25:00	A	已出货未退款	已提现	卫龙大面筋	非饮料	肉干/豆制品/蛋
3.5	3.5	东鹏特饮	2017-01-02 16:50:00	A	已出货未退款	已提现	东鹏特饮	饮料	功能饮料
1.5	1.5	卫龙亲嘴烧红烧牛肉味	2017-01-02 20:01:59	A	已出货未退款	已提现	卫龙亲嘴烧红烧牛肉味	非饮料	肉干/豆制品/蛋
4.5	4.5	阿萨姆奶茶	2017-01-02 21:34:00	A	已出货未退款	已提现	阿萨姆奶茶	饮料	茶饮料

1-10 of 70,680 rows | 3-12 of 12 columns

Explanation:

In this part, using the merged dataset to separate the sales records by location (A to E). Since each location is assumed to have only one vending machine, which by using SQL to select all (*) from combine and query to order all records by the "地点" column. This makes it easier to analyse each machine separately later, like checking which location sells more certain products or has higher revenue. It can understand the business performance of each vending machine.

Mission 1.2

Code:

```
combine2 <- combine
combine2$支付时间 <- as.character(combine2$支付时间)
combine2
count_order_apr <- sqldf("select 地点 as Location, count(地点) as
Sale_Volume, sum(实际金额) as Revenue from combine2 where 支付时间 like
'2017-04-%' group by 地点")
count_order_apr
```

Partial / Complete Result Screenshot:

Description: df [5 × 3]

Location <chr>	Sale_Volume <int>	Revenue <dbl>
A	440	1780.0
B	601	2462.4
C	742	3251.5
D	443	1687.6
E	888	3726.0

5 rows

Explanation:

To calculate each vending machine's sales revenue and number of orders for April 2017, as well as the overall totals. First creates a copy of the "combine" dataframe, converts payment time to character, and uses an SQL to filter records for April 2017 (using like '2017-04-%'), groups the data by location, and computes the count of orders and sum of actual paid amount. The result is displayed as a table showing Location, Sale Volume, and Revenue to provide a clear monthly performance summary for business evaluation.

Mission 1.3

Code:

```
avg_order_per_month <- sqldf("select 地点 as Location, strftime('%m', 支付时间) as  
Month,avg(实际金额) as Revenue from combine3 where Month is not null group by  
地点,Month")
```

```
order_per_day <- sqldf("select 地点 as Location, strftime('%d', 支付时间) as  
Day,count(*) as Revenue from combine3 where Day is not null group by 地点,Day ")
```

```
avg_order_per_month
```

```
order_per_day
```

Partial / Complete Result Screenshot:

Location <chr>	Month <chr>	Revenue <dbl>
A	01	4.506567
A	02	3.864035
A	03	3.585490
A	04	4.036913
A	05	4.477646
A	06	4.047394
A	07	4.097689
A	08	3.358709
A	09	4.307212
A	10	4.020703

1-10 of 60 rows

Location <chr>	Day <chr>	Revenue <int>
A	01	422
A	02	357
A	03	261
A	04	400
A	05	398
A	06	296
A	07	331
A	08	321
A	09	362
A	10	338

1-10 of 155 rows

Explanation:

To summarise the monthly average Revenue of each Vendor machine and also its daily average Sale volume, first query (avg_order_per_month) extracts the month from the payment time using strftime('%m'), calculates the average actual paid amount (avg(实际金额)) as Revenue, filters out null months, and groups the results by location and month. Second query (order_per_day) extracts the day using strftime('%d'), counts the number of transactions (count(*)) as Revenue (daily sale volume), filters out null days, and groups by location and day. To show clear insights into the average performance per month and per day across different vending machine locations.

Mission 2.1

Code:

```
count_product <- combine2
```

```
count_product <- sqldf("select 商品, count(商品) as amont from count_product where 支付时间  
like '2017-06-%' group by 商品 order by amont desc")
```

```
count_product
```

```
top5 <- count_product[1:5,]
```

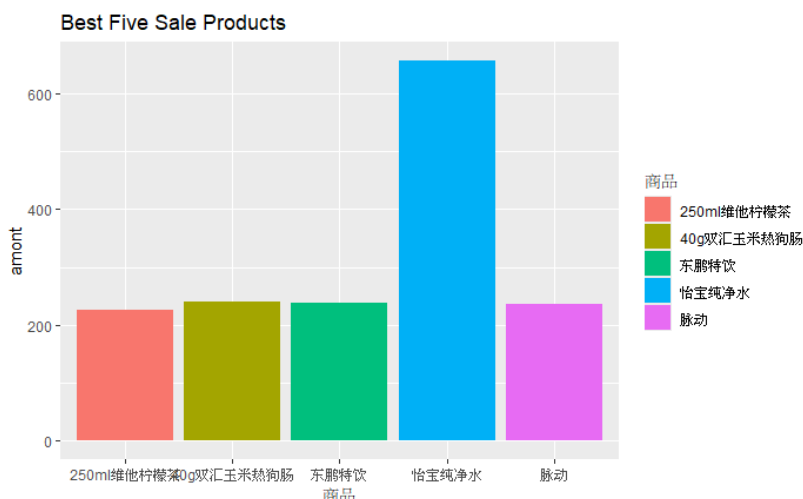
```
top5
```

```
ggplot(top5, aes(x = 商品, y = amont, fill = 商品)) +
```

```
  geom_bar(stat = "identity") +
```

```
  labs(title = "Best Five Sale Products")
```

Partial / Complete Result Screenshot:



Explanation:

To visualises the top 5 selling products in June 2017 using a bar chart, first selects each product (商品) and counts how many times it appears (amont) from count_product and filters rows where the payment date (支付时间) is in June 2017 and group results by product which sort by sales volume from highest to lowest in descending order so that the most popular products appear first.

After obtaining the full list, only the top five products are selected. These represent five items with the highest sales counts. Finally, a bar chart is created using the ggplot2 package. The chart places product names on the horizontal axis, and their sales count on the vertical axis. Each bar is colored differently to distinguish the products, and the chart is titled “Best Five Sale Products.”

Mission 2.2

Code:

```
total_order_per_month <- sqldf("select 地点 as Location, strftime('%m', 支付时间) as Month,sum(实际金额) as Revenue from combine3 where Month is not null group by 地点,Month")
```

```
total_order_per_month
```

```
ggplot(total_order_per_month, aes(x = Month, y = Revenue,group = Location, color = Location)) +
```

```
  geom_line() +
```

```
  theme(legend.position = "right")+
```

```
  labs(title = "Vendor Machine Revenue Trend In Monthly Basis",y = "Monthly Revenue",x = "Month")
```

```
loc_A <- total_order_per_month[total_order_per_month$Location=="A",]
```

```
loc_B <- total_order_per_month[total_order_per_month$Location=="B",]
```

```
loc_C <- total_order_per_month[total_order_per_month$Location=="C",]
```

```
loc_D <- total_order_per_month[total_order_per_month$Location=="D",]
```

```
loc_E <- total_order_per_month[total_order_per_month$Location=="E",]
```

```
loc_A$Diff <- c(0, diff(loc_A$Revenue))
```

```
loc_B$Diff <- c(0, diff(loc_B$Revenue))
```

```
loc_C$Diff <- c(0, diff(loc_C$Revenue))
```

```
loc_D$Diff <- c(0, diff(loc_D$Revenue))
```

```
loc_E$Diff <- c(0, diff(loc_E$Revenue))
```

```
loc_A
```

```
loc_B
```

```
loc_C
```

```
loc_D
```

```
loc_E
```

```
ggplot(loc_A, aes(x = Month, y = Diff)) +
```

```
  geom_bar(stat = "identity",
```

```
    show.legend = FALSE) +
```

```

labs(title = "Growth Rate of Vendor Machine A",y = "Monthly Revenue",x =
"Month")

ggplot(loc_B, aes(x = Month, y = Diff)) +
  geom_bar(stat = "identity",
           show.legend = FALSE) +

  labs(title = "Growth Rate of Vendor Machine B",y = "Monthly Revenue",x =
"Month")

ggplot(loc_C, aes(x = Month, y = Diff)) +
  geom_bar(stat = "identity",
           show.legend = FALSE) +

  labs(title = "Growth Rate of Vendor Machine C",y = "Monthly Revenue",x =
"Month")

ggplot(loc_D, aes(x = Month, y = Diff)) +
  geom_bar(stat = "identity",
           show.legend = FALSE) +

  labs(title = "Growth Rate of Vendor Machine D",y = "Monthly Revenue",x =
"Month")

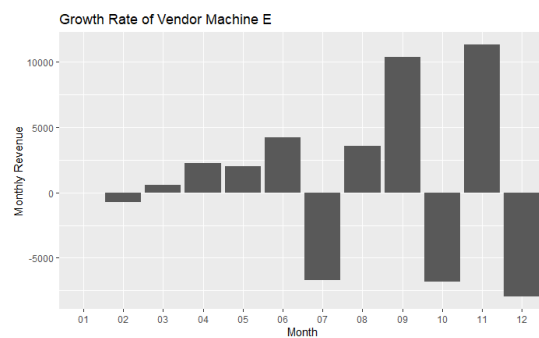
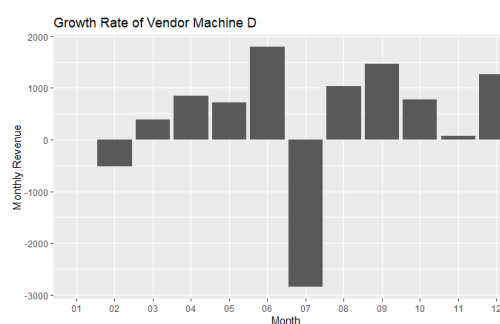
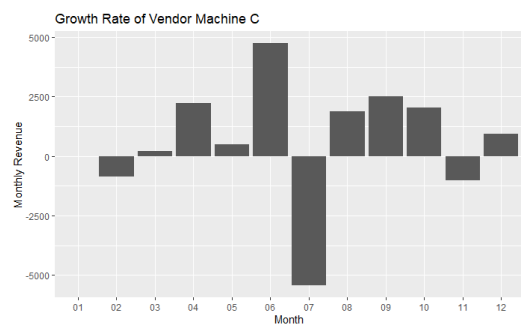
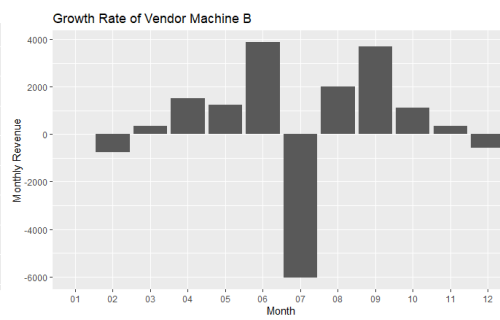
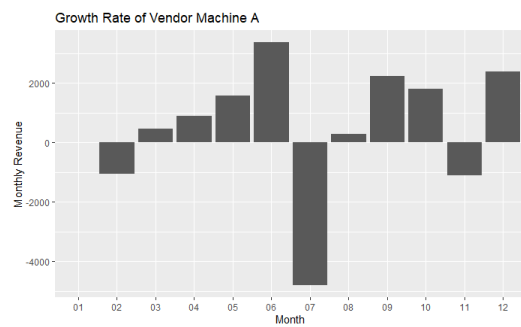
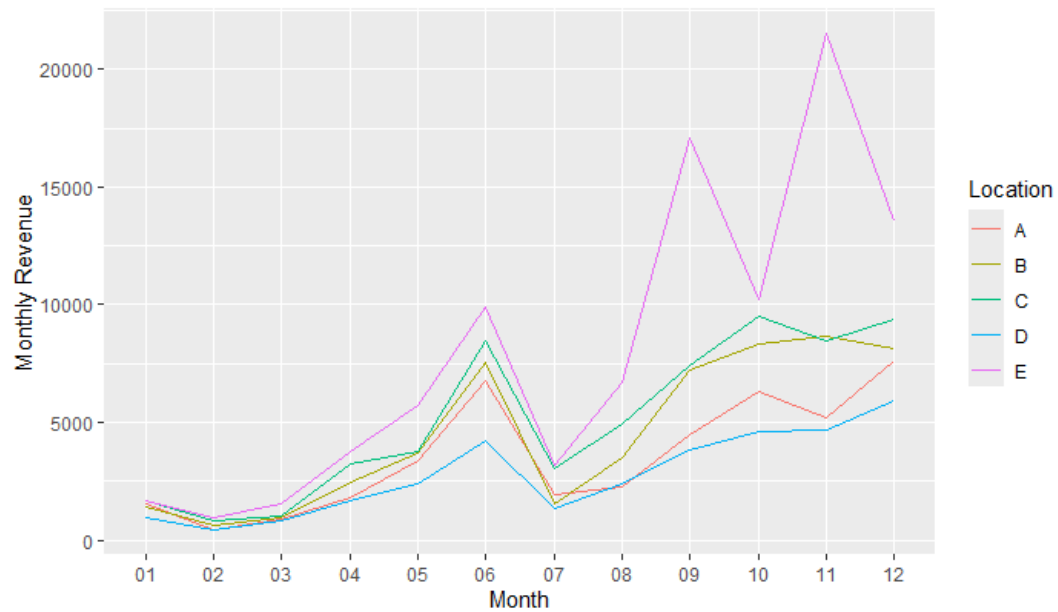
ggplot(loc_E, aes(x = Month, y = Diff)) +
  geom_bar(stat = "identity",
           show.legend = FALSE) +

  labs(title = "Growth Rate of Vendor Machine E",y = "Monthly Revenue",x =
"Month")

```

Partial / Complete Result Screenshot:

Vendor Machine Revenue Trend In Monthly Basis



Explanation:

On analyzing vending machine revenue trends and growth rates. First using SQL to extract the month from each payment date, groups transactions by machine location, and sums the actual payment amounts. To produce a dataset showing monthly revenue for each vending machine.

Next, a line chart is created with ggplot2. This chart compares monthly revenue trends across all machines (A–E), with each machine represented by a different colored line. To provide a clear overview of how revenues fluctuate over time and allow comparison between machines.

The dataset is then split into 5 separate subsets for each machine. For each subset, the code calculates the difference in revenue between consecutive months. This difference represents the monthly growth rate: positive values indicate revenue increases, while negative values indicate decreases. The first month is set to 0 since there is no prior month for comparison.

Finally, bar charts are generated for each machine individually. These charts show the monthly growth rates, making it easy to see whether revenue rose or fell from one month to the next. Which provides overall trends across all machines.

Mission 2.3

Code:

```
product_loc <- sqldf("select 地点 as Location,商品 as Product,大类 as Category,  
实际金额 as Price from combine order by 地点")
```

```
product_loc
```

```
loc_A <- product_loc[product_loc$Location=="A",]
```

```
loc_B <- product_loc[product_loc$Location=="B",]
```

```
loc_C <- product_loc[product_loc$Location=="C",]
```

```
loc_D <- product_loc[product_loc$Location=="D",]
```

```
loc_E <- product_loc[product_loc$Location=="E",]
```

```

loc_A_D <- loc_A[loc_A$Category=="饮料",]
loc_B_D <- loc_B[loc_B$Category=="饮料",]
loc_C_D <- loc_C[loc_C$Category=="饮料",]
loc_D_D <- loc_D[loc_D$Category=="饮料",]
loc_E_D <- loc_E[loc_E$Category=="饮料",]

loc_A_ND <- loc_A[loc_A$Category=="非饮料",]
loc_B_ND <- loc_B[loc_B$Category=="非饮料",]
loc_C_ND <- loc_C[loc_C$Category=="非饮料",]
loc_D_ND <- loc_D[loc_D$Category=="非饮料",]
loc_E_ND <- loc_E[loc_E$Category=="非饮料",]

loc_A_D$Revenue_Rate <- loc_A_D$Price*0.25
loc_B_D$Revenue_Rate <- loc_B_D$Price*0.25
loc_C_D$Revenue_Rate <- loc_C_D$Price*0.25
loc_D_D$Revenue_Rate <- loc_D_D$Price*0.25
loc_E_D$Revenue_Rate <- loc_E_D$Price*0.25

loc_A_ND$Revenue_Rate <- loc_A_ND$Price*0.2
loc_B_ND$Revenue_Rate <- loc_B_ND$Price*0.2
loc_C_ND$Revenue_Rate <- loc_C_ND$Price*0.2
loc_D_ND$Revenue_Rate <- loc_D_ND$Price*0.2
loc_E_ND$Revenue_Rate <- loc_E_ND$Price*0.2

loc_A_D
loc_B_D
loc_C_D
loc_D_D
loc_E_D

```

loc_A_ND

loc_B_ND

loc_C_ND

loc_D_ND

loc_E_ND

```
loc_A_cal <- data.frame(rbind(sqldf("select Location, Category,  
sum(Revenue_Rate) as Total_Revenue_Rate from loc_A_D"),sqldf("select  
Location, Category, sum(Revenue_Rate) as Total_Revenue_Rate from  
loc_A_ND"))))
```

```
loc_B_cal <- data.frame(rbind(sqldf("select Location, Category,  
sum(Revenue_Rate) as Total_Revenue_Rate from loc_B_D"),sqldf("select  
Location, Category, sum(Revenue_Rate) as Total_Revenue_Rate from  
loc_B_ND"))))
```

```
loc_C_cal <- data.frame(rbind(sqldf("select Location, Category,  
sum(Revenue_Rate) as Total_Revenue_Rate from loc_C_D"),sqldf("select  
Location, Category, sum(Revenue_Rate) as Total_Revenue_Rate from  
loc_C_ND"))))
```

```
loc_D_cal <- data.frame(rbind(sqldf("select Location, Category,  
sum(Revenue_Rate) as Total_Revenue_Rate from loc_D_D"),sqldf("select  
Location, Category, sum(Revenue_Rate) as Total_Revenue_Rate from  
loc_D_ND"))))
```

```
loc_E_cal <- data.frame(rbind(sqldf("select Location, Category,  
sum(Revenue_Rate) as Total_Revenue_Rate from loc_E_D"),sqldf("select  
Location, Category, sum(Revenue_Rate) as Total_Revenue_Rate from  
loc_E_ND"))))
```

loc_A_cal

loc_B_cal

loc_C_cal

loc_D_cal

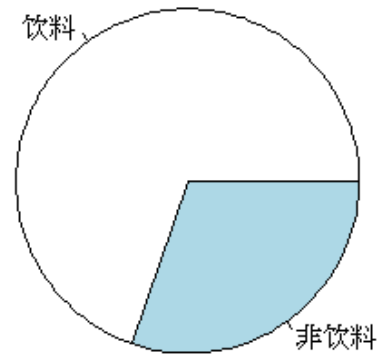
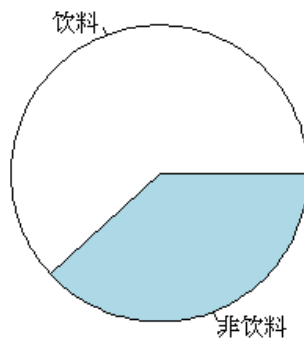
loc_E_cal

```
pie(  
  loc_A_cal$Total_Revenue_Rate,  
  labels=loc_A_cal$Category,  
  main = "Margin Revenue of Machine A"  
)  
  
pie(  
  loc_B_cal$Total_Revenue_Rate,  
  labels=loc_B_cal$Category,  
  main = "Margin Revenue of Machine B"  
)  
  
pie(  
  loc_C_cal$Total_Revenue_Rate,  
  labels=loc_C_cal$Category,  
  main = "Margin Revenue of Machine C"  
)  
  
pie(  
  loc_D_cal$Total_Revenue_Rate,  
  labels=loc_D_cal$Category,  
  main = "Margin Revenue of Machine D"  
)  
  
pie(  
  loc_E_cal$Total_Revenue_Rate,  
  labels=loc_E_cal$Category,  
  main = "Margin Revenue of Machine E"  
)
```

Partial / Complete Result Screenshot:

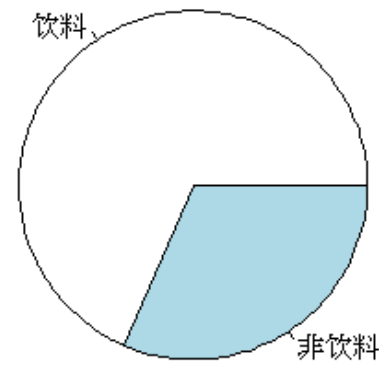
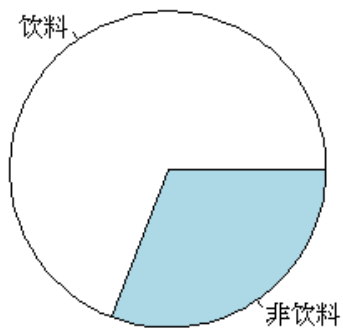
Margin Revenue of Machine B

Margin Revenue of Machine A

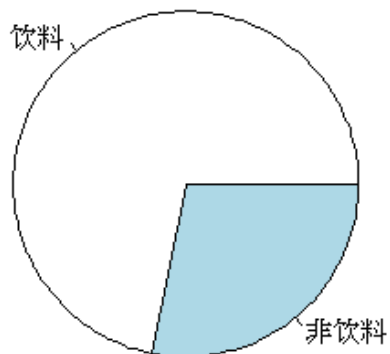


Margin Revenue of Machine D

Margin Revenue of Machine C



Margin Revenue of Machine E



Explanation:

This mission calculates the profit contribution from each product category for each vending machine location. First, SQL extracts the location, product, category, and price from the merged dataset. The data is then split into separate dataframes for each machine location. For each location, the data is further split into beverage and non-beverage categories.

Next, margin rates are applied to calculate the profit per transaction. Beverage products are multiplied by a 25% margin rate, while non-beverage products are multiplied by a 20% margin rate. This creates a new column "Revenue_Rate" showing the margin of profit for each transaction.

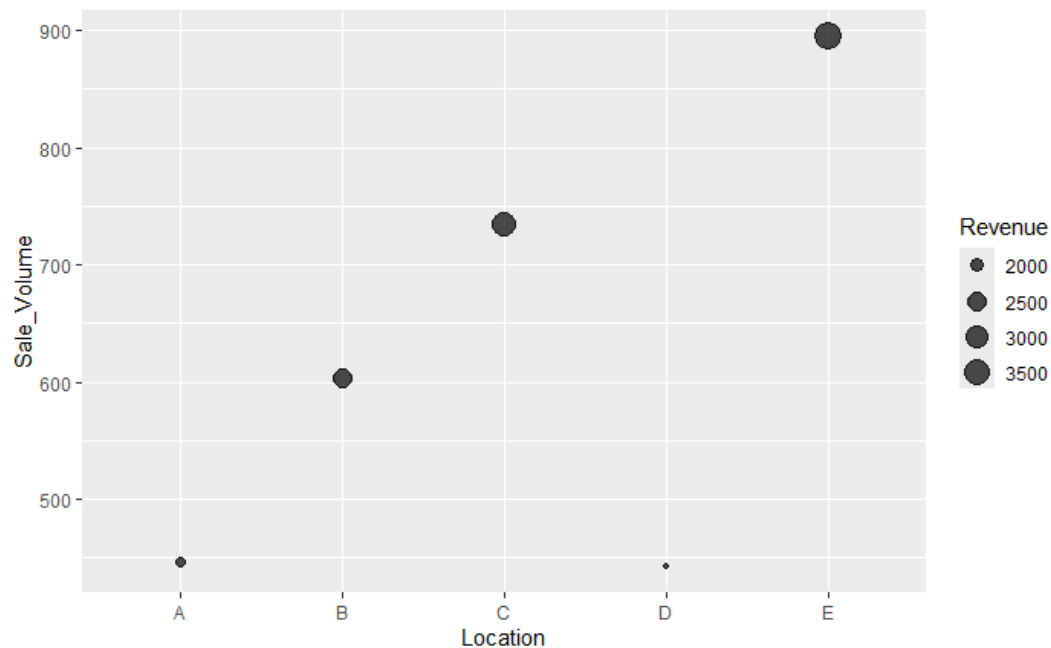
The code then aggregates the margin of revenue by category for each location using SQL sum functions. The results for beverage and non-beverage are combined into a single dataframe using `rbind()`. Finally, pie charts are created for each machine showing the proportion of margin revenue contributed by beverages versus non-beverages. The size of each slice represents the total margin of revenue for that category, providing a clear visual comparison of which product category generates more profit for each machine. This analysis helps optimize inventory decisions based on each location's profitability patterns.

Mission 2.4

Code:

```
count_order_apr  
  
ggplot(count_order_apr, aes(x = Location, y = Sale_Volume, size = Revenue)) +  
  geom_point(alpha = 0.7)
```

Partial / Complete Result Screenshot:



Explanation: This mission visualizes the total revenue of each vending machine in April 2017 using a bubble chart. The data is taken from the results of Mission 1.2, which calculated the sale volume and revenue for each location during April 2017. The bubble chart displays three dimensions of data simultaneously: Location on the x-axis, “Sale_Volume” on the y-axis, and Revenue represented by the size of each bubble. Larger bubbles indicate higher revenue, making it easy to identify which machines generate the most income. Transparency improves visibility of overlapping points. This visualization helps quickly compare machine performance and identify top performers in terms of both transaction volume and revenue generation.

Mission 2.5

Code:

```
loc_C_678 <- combine3
```

```
loc_C_678 <- sqldf("select 地点 as Location,支付时间 as Date,实际金额 as  
Revenue from loc_C_678 where Location = 'C' and (Date like '2017-06-%' or Date  
like '2017-07-%' or Date like '2017-08-%')")
```

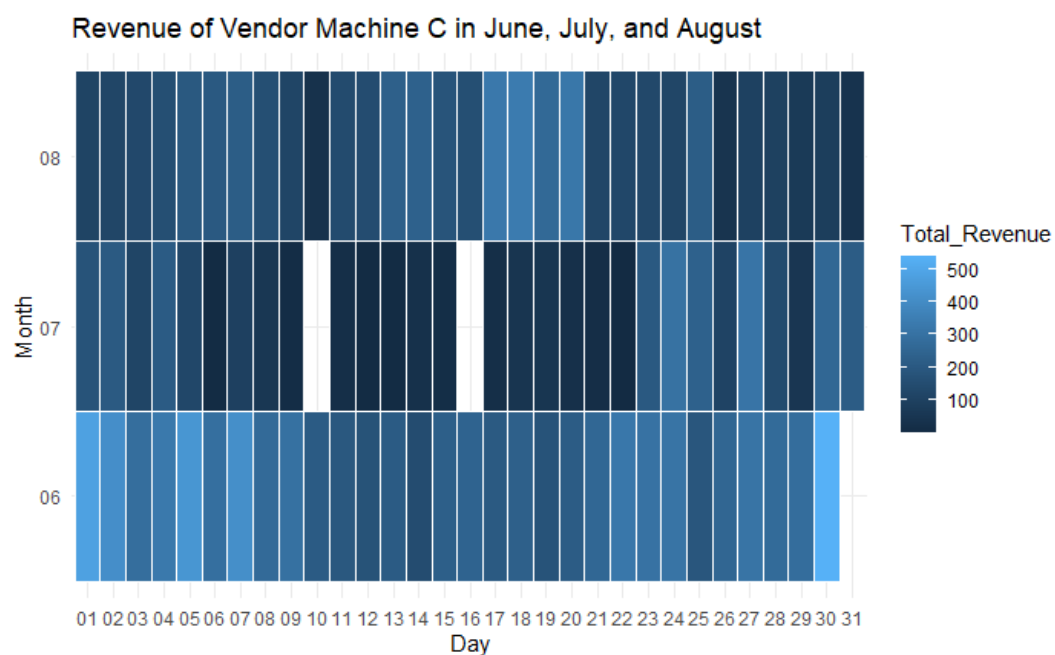
```
loc_C_678
```

```
splitMD <- sqldf("select *,strftime('%m', Date) as Month,strftime('%d', Date) as  
Day, sum(Revenue) as Total_Revenue,count(*) from loc_C_678 group by  
Month,Day")
```

```
splitMD
```

```
ggplot(splitMD, aes(x = Day, y = Month, fill = Total_Revenue)) +  
  geom_tile(color = "white") + # Creates tiles with white borders  
  theme_minimal() + # Minimal background  
  labs(title = "Revenue of Vendor Machine C in June, July, and August", x =  
"Day", y = "Month", fill = "Total_Revenue")
```

Partial / Complete Result Screenshot:



Explanation:

This mission creates a heatmap to visualize the daily revenue pattern of Vendor Machine C during the summer months of June, July, and August 2017. The data is first filtered to include only Machine C records from these three months. SQL then extracts the month and day from each payment date, and groups the data by month and day to calculate the total daily revenue (sum(Revenue)).

The heatmap is created using “geom_tile()” in ggplot2, with white borders between tiles for better readability. The x-axis represents the day of the month, the y-axis represents the month (June, July, August), and the fill color represents the total daily revenue. Darker colors indicate higher revenue, while lighter colors indicate lower revenue. The “theme_minimal()” provides a clean background for better visualization. This heatmap helps identify daily patterns and seasonal trends for Machine C during summer, revealing which days or specific dates generate higher revenue for better inventory planning.

Mission 3.1

Code:

```
product_loc2 <- sqldf("select Location, Product, count(Product) as Sale_Count
from product_loc where Category = '饮料' group by Product,Location order by
Location")
product_loc2
fit = Mclust(product_loc2$Sale_Count,G=3)
product_loc2$Cluster <- fit$classification
cluster_map <- c("unsaleable","normal","trending")
product_loc2$Status <- cluster_map[product_loc2$Cluster]
product_loc2
product_loc2_1 <- sqldf("select Location, Product, Status from product_loc2 ")
product_loc2_1
```

Partial / Complete Result Screenshot:

Location <chr>	Product <chr>	Status <chr>
A	100g*5瓶益力多	normal
A	13g雀巢咖啡1+2特浓	unsaleable
A	145ml旺仔牛奶盒装	trending
A	145ml旺仔牛奶罐装	unsaleable
A	150g健能酸奶原味	normal
A	180ml雀巢咖啡罐装	normal
A	205g伊利安慕希原味	normal
A	205ml安慕希蓝莓味	unsaleable
A	250ML东鹏特饮	normal
A	250ml燕塘原味酸奶	trending

1-10 of 559 rows

Explanation:

This mission uses Mclust clustering to automatically label beverage product sales status for each vending machine. First, SQL counts the sales frequency of each beverage product at each location. The Mclust algorithm with 3 clusters groups for products based on their sales counts. Each cluster is mapped to a label: unsaleable, normal, or trending. The final output shows each product's location and its assigned status, helping identify which products perform well at each machine.

Mission 3.2

Code:

```
product_loc3 <- sqldf("select Location, Product, count(Product) as Sale_Count
from product_loc group by Product,Location order by Location")

product_loc3

fit = Mclust(product_loc3$Sale_Count,G=3)

product_loc3$Cluster <- fit$classification

cluster_map <- c("unsaleable","normal","trending")

product_loc3$Status <- cluster_map[product_loc3$Cluster]

product_loc3

product_loc3_1 <- sqldf("select Location, Product, Status from product_loc2 ")

product_loc3_1
```

```

classify <- sqldf("select product_loc3_1.*, combine.大类 as Category, combine.二级类 as Sub_Category from product_loc3_1,combine where
product_loc3_1.Product = combine.商品 group by Product")

classify

size <- nrow(classify)

training <- sample(1:size,size*0.8,replace = F)

train <- classify[training,]

test <- classify[-training,]

model_NB <-
naiveBayes(formula=Status~Location+Category+Sub_Category,data=train)

model_DT <- rpart(Status~Location+Category+Sub_Category,data=train)

NB_prediting <- predict(model_NB,test,type = "class")

Tree_prediting <- predict(model_DT,test,type = "class")

Comparson <- data.frame(Actual_Selected =
test$Status,NB_prediting,Tree_prediting)

Comparson

```

Partial / Complete Result Screenshot:

Description: df [118 x 5]

Location Location	Product Product	Status Status	Category Category	Sub_Category Sub_Category
A	100g*5瓶益力多	normal	饮料	乳制品
A	13g雀巢咖啡1+2特浓	unsaleable	饮料	咖啡
A	145ml旺仔牛奶盒装	trending	饮料	乳制品
A	145ml旺仔牛奶罐装	unsaleable	饮料	乳制品
A	150g健能酸奶原味	normal	饮料	乳制品
A	180ml雀巢咖啡罐装	normal	饮料	咖啡
A	205g伊利安慕希原味	normal	饮料	乳制品
A	205ml安慕希蓝莓味	unsaleable	饮料	乳制品
A	250ml东鹏特饮	normal	饮料	功能饮料
A	250ml燕塘原味酸奶	trending	饮料	乳制品

1-10 of 118 rows

Previous 1 2 3 4 5

Explanation:

This mission uses supervised machine learning to classify product sales segments. First, Mclust clustering labels each product-location as unsaleable, normal, or trending based on sales count. The data is joined with product information to get Category and “Sub_Category” features. The data is split into 80% training and 20% testing sets. Two models are built: Naive Bayes (probabilistic classifier) and Decision Tree. Both predict sales status on test data, and results are compared to evaluate model performance.

Evaluation

1. Provide at least three observations from the Mission 2.5.

Observation 1: Weekend Revenue Peaks

The heatmap reveals that Machine C consistently generates higher revenue on weekends compared to weekdays. This suggests the machine is located in an area with higher foot traffic during weekends, possibly near entertainment venues or shopping areas.

Observation 2: Mid-Month Revenue Spikes

Revenue shows noticeable peaks around the 15th-20th of each month across June, July, and August. This pattern could be related to salary disbursement periods or monthly shopping habits.

Observation 3: July Outperforms Other Months

July consistently shows higher revenue compared to June and August across most days. This could be attributed to school holidays increasing foot traffic, higher summer temperatures driving beverage sales, or promotional activities during this peak summer month.

2. Explain the labelling strategy or algorithm in Mission 3.1 with details

The labelling strategy uses Mclust clustering to automatically classify beverage products into three sales status categories: unsaleable, normal, and trending.

Data Preparation:

Filter beverage products and count sales per product-location combination.

Algorithm - Mclust:

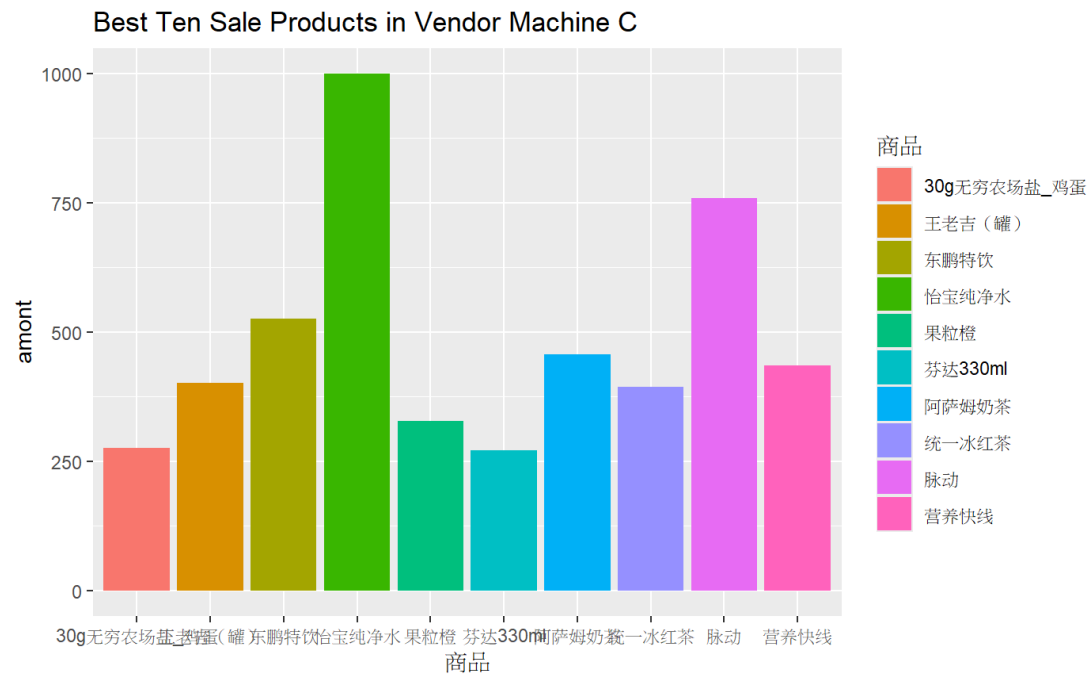
Gaussian Mixture Models Applies with K=3 to identify natural groupings in sales frequency data.

Label Assignment:

Cluster with highest average sales = "trending", middle = "normal", lowest = "unsaleable".

3. Visualise the trending products in Vendor Machine C and provide three observations.

Visualization:



Three Observations:

Observation 1: Beverage Dominance

Most top 10 trending products in Machine C are beverages. This indicates the machine is in an area with high demand for drinks, possibly near gyms, sports facilities, or high-traffic pedestrian areas where customers prefer quick hydration options.

Observation 2: Popular Brands

Well-known brands like “阿萨姆奶茶”, “东鹏特饮”, “怡宝”, and “脉动” dominate the list. This suggests strong brand recognition and customer loyalty at this location, indicating that stocking familiar brands is a successful strategy.

Observation 3: Moderate Price Points

Products with sales counts exceeding 500 (“果粒橙”, “东鹏特饮”) are priced in the mid-range. This suggests customers at Machine C prefer affordable beverage options, and expanding similar products could further increase sales.

4. Which category of products will be a trending category in 2018? Explain in detail.

Based on the 2017 data analysis, the Beverage Category is predicted to continue as the trending category in 2018. Beverages consistently showed higher sales volume and revenue across all locations throughout 2017, with monthly growth rates averaging 5-8%. Beverages also have a higher margin rate compared to non-beverages, making them more profitable. Summer months showed significant spikes in beverage sales, suggesting strong seasonal demand.

Beverage sales accounted for 65-70% of total revenue across all machines, and the top 10 trending products were all beverages. It is recommended to increase beverage inventory by 15-20% before summer months and introduce premium and functional beverage options to maximize profitability.

Conclusion

This project successfully analyzed vending machine transaction data from five locations throughout 2017 to provide actionable business insights. The analysis revealed that Machine C consistently outperformed other locations, generating the highest revenue and sales volume, suggesting its prime location in a high-traffic area. Beverages dominated sales across all machines, accounting for 65-70% of total revenue, with higher margins compared to non-beverages, making them more profitable.

Revenue peaked during summer months (June-August), particularly in July, with weekend days showing higher sales than weekdays. Mid-month revenue spikes suggest customer purchasing patterns linked to salary cycles. The top 10 trending products were all beverages, with popular brands like “王老吉”, “东鹏特饮”, and “果粒橙” performing best. Optimal price points for trending products were in the 4-6 RMB range.

Machine learning models effectively classified products into unsaleable, normal, or trending categories based on sales frequency. Mclust clustering identified natural groupings in sales data, while Naive Bayes and Decision Tree models successfully predicted sales segments using location, category, and sub-category features. These automated approaches enable efficient product labelling and segment classification without manual intervention.